

Tema 12

Eventos, delegados y funciones Lambda



Índice

- Introducción
- Funciones Lambda
- Delegados
- Eventos
- Programación funcional en C#
- Diferencias y ventajas de cada tipo
- Buenas prácticas para el uso de Lambdas, delegados y eventos



- En C#, los delegados, eventos y funciones lambda son herramientas fundamentales para trabajar con programación basada en funciones, y nos permiten escribir código flexible y reutilizable.
- Cada uno de estos conceptos desempeña un papel específico.

1. Introducción



- **Delegados:** Son tipos que representan referencias a métodos y permiten definir variables que pueden almacenar y ejecutar métodos de forma dinámica.
- Los delegados proporcionan una manera de pasar métodos como parámetros y definir comportamiento en tiempo de ejecución, lo cual es especialmente útil en eventos y otros patrones de diseño.

1. Introducción

- **Eventos:** Los eventos son una forma de notificación que permite a una clase informar a otras clases cuando ocurre un cambio o una acción específica.
- Los eventos en C# están basados en delegados y son ampliamente usados en la programación orientada a eventos, donde los componentes reaccionan a acciones, como clics de botones o cambios en los datos.

1. Introducción

¿Por qué usar Delegados, Eventos y Lambdas?

Estos elementos son esenciales en C# porque ayudan a:

- **Abstraer el comportamiento:** Los delegados y lambdas permiten que el comportamiento del código sea parametrizable.

1. Introducción

- **Simplificar el código:** Las funciones lambda ofrecen una forma compacta de definir operaciones complejas, especialmente en consultas LINQ y en la programación funcional.
- **Implementar patrones de diseño:** Los eventos son clave para implementar patrones como el *Observer*, que permite que un objeto notifique a otros sobre cambios de estado.

1. Introducción

Ejemplo teórico

- Un escenario común que involucra delegados, eventos y lambdas es el manejo de un botón en una interfaz gráfica de usuario.
- Al hacer clic en el botón, se dispara un **evento** que está **vinculado a un delegado**, el cual ejecuta un método en respuesta a la acción.

1. Introducción

- Si queremos definir rápidamente el método a ejecutar, podemos usar una **expresión lambda** que simplifique el código, en lugar de crear un método adicional para manejar el evento.
- Este enfoque hacia la programación funcional y orientada a eventos hace que el código sea más modular y fácil de mantener.

1. Introducción

- Las **funciones Lambda** en C# son una forma compacta y simplificada de definir funciones anónimas.
- Estas expresiones Lambda son muy útiles en programación funcional y permiten crear funciones breves que se pueden pasar como parámetros a otros métodos.

2. Funciones Lambda

- La estructura básica de una expresión lambda es la siguiente:
- Si el cuerpo de la lambda requiere más de una línea, se puede usar un bloque de código con llaves {}:

2. Funciones Lambda

```
(parameters) => expr  
(parameters) =>  
{  
    // Múltiples líneas  
    return value;  
}
```

- En el tema anterior vimos muchos de estos ejemplos sin saber qué eran realmente: muchas de las consultas LINQ que creábamos estaban compuestas por una función Lambda.
- A lo largo de este tema también vamos a trabajar con funciones Lambda.

2. Funciones Lambda

- Por ejemplo, cuando expresábamos una condición en la consulta Where:

- En este caso, `n => n % 2 == 0` es una función Lambda.

2. Funciones Lambda

```
1 List<int> numbers = new List<int> { 1, 2, 3, 4 }
2 var evenNumbers = numbers.Where(n => n % 2 == 0)
3
4 Console.WriteLine(string.Join(", ", evenNumbers));
```

- Las funciones Lambdas pueden capturar variables locales de su entorno, lo que significa que pueden hacer referencia a variables externas a la Lambda.
- Esto se conoce como "**captura de variables**"
- En este ejemplo, la Lambda (`x => x * multiplier`) captura la variable `multiplier`.

2. Funciones Lambda

```
1 int multiplier = 5;  
2 Func<int, int> multiply = x => x * multiplier;  
3  
4 Console.WriteLine(multiply(10)); // 50
```

Ventajas de las Expresiones Lambda

- **Código más conciso:** Eliminan la necesidad de crear métodos separados para cada pequeña operación.
- **Legibilidad:** Para tareas simples, una lambda es más fácil de leer en línea que un método completo.
- **Flexibilidad:** Las lambdas permiten escribir funciones en el lugar exacto donde se usan, ideal para operaciones rápidas.

2. Funciones Lambda

Vídeo práctico

2. Funciones Lambda

Vídeo práctico - código

2. Funciones Lambda

```
1 List<int> list = new List<int>() { 1, 2, 3, 4, 5, 6, 7 }  
2
```

- En C#, un **delegado** es un tipo que representa una referencia a un método.
- Los delegados permiten almacenar **métodos en variables**, pasar métodos como parámetros y definir comportamientos en tiempo de ejecución.
- Son útiles para implementar el comportamiento de métodos de manera flexible y permiten técnicas avanzadas como la programación basada en eventos.

3. Delegados

Sintaxis de Delegados

- La declaración de un delegado especifica la **firma del método** al que puede apuntar.
- Es decir, especifica los tipos de parámetros y el tipo de retorno que debe tener cualquier método que se asigne a ese delegado.

3. Delegados

Ejemplo

- En este ejemplo, `Operation` es un **tipo delegado** que representa **cualquier método** que reciba dos enteros (`int x` e `int y`) y devuelva un entero (`int`).

3. Delegados

```
public delegate int Operation(int
```

Uso Básico de Delegados

Para usar un delegado, es necesario:

1. **Declararlo** con la firma del método.
2. Crear una **instancia** del delegado, asignándole un método que cumpla con su firma.
3. **Invocar** el delegado como si fuera el método asignado.

3. Delegados

Ejemplo

3. Delegados

```
1  public class Calculator
2  {
3      public int Add(int a, int b)
4      {
5          return a + b;
6      }
7
8      public int Subtract(int a, int b)
9      {
10         return a - b;
11     }
12 }
13
14 public class Program
15 {
16     public delegate int Operation(int x, int y); // Def
17
18     public static void Main()
19     {
20         Calculator calculator = new Calculator();
21     }
```

- En el ejemplo anterior, hemos declarado un delegado `Operation` dentro de la clase `Program`.
- La clase `Calculator` define 2 métodos que cumplen la firma del delegado.
- Creamos una instancia del delegado (`Operation operation`) y le asignamos uno de los métodos (`operation = calculator.Add`).

3. Delegados

- Invocamos al delegado (`operation(5, 3)`), con lo que se ejecuta el método que tiene asignado (`calculator.Add`).
- Posteriormente cambiamos la asignación del delegado (`operation = calculator.Subtract`) y volvemos a invocar al delegado (`operation(5, 3)`), que esta vez ejecutará el nuevo método asignado (`calculator.Subtract`)

3. Delegados

Vídeo práctico

3. Delegados

Vídeo práctico - código Program

3. Delegados

```
1 public class Program
2 {
3     public delegate string MessageDelegate(string message);
4     public delegate void Callback(int x, int y);
5
6     public static void Main()
7     {
8         Car car = new Car();
9         Person person = new Person();
10
11         MessageDelegate message = car.ShowInfo;
12         string carInfo = message("Fiesta", 1992);
13
14         Console.WriteLine(carInfo);
15
16         message = person.GetId;
17         string personId = message("Antonio", 46846135);
18
19         Console.WriteLine(personId);
20     }
```

Vídeo práctico - código Car / Person

3. Delegados

```
1  internal class Car
2  {
3      public string ShowInfo(string model, int year)
4      {
5          return string.Format("El coche es un {0} del año {1}", model, year);
6      }
7
8      public void GetPower(int hp, int cc)
9      {
10         Console.WriteLine("El coche tiene {0} caballos de potencia", hp);
11     }
12 }
13
14 internal class Person
15 {
16     public string GetId(string name, int id)
17     {
18         return string.Format("El DNI de la persona {0} es {1}", name, id);
19     }
20 }
```

Delegados Genéricos

- En C#, existe el concepto de delegados genéricos que ofrecen una forma flexible de trabajar con delegados sin necesidad de crear tipos específicos.
- Son ampliamente utilizados en LINQ, expresiones lambda y programación funcional.

3. Delegados

- Estos delegados genéricos son `Action`, `Func` y `Predicate` y resultan útiles para escribir código reutilizable.
- Al igual que cuando declarábamos colecciones genéricas podíamos especificar un tipo (`ICollection<T>`), en el caso de los delegados genéricos vamos a poder especificar los tipos de parámetros de entrada (y salida en el caso de `Func`).

3. Delegados

Action

- El delegado Action representa un método que realiza una acción sin devolver ningún valor (void).
- Puede tomar entre 0 y 16 parámetros de entrada.

3. Delegados

```
1 // Action que toma un parámetro de tipo string y no devuelve
2 Action<string> greetings =
3     name => Console.WriteLine(string.Format("¡Hola, {0}!", na
4
5 greetings("Manuel"); // Salida ¡Hola, MANUEL!
```

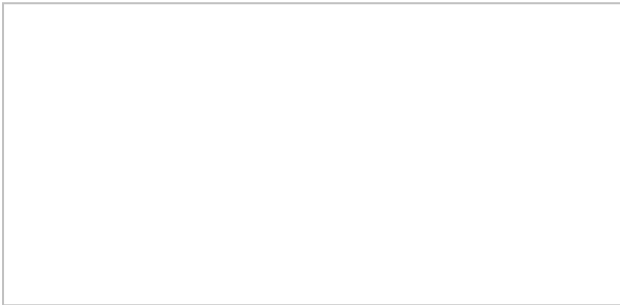
- Los `Action` son especialmente útiles para pasar lógica que solo requiere ejecutar una operación sin devolver un resultado.
- Por ejemplo, en un bucle `foreach`, podemos iterar sobre cada elemento mediante un `Action`:

3. Delegados

```
1 List<int> numbers = new List<int> {1, 2, 3, 4, 5};  
2 Action<int, int> printNumber = (n, factor) => Console.WriteLine  
3  
4 numbers.ForEach(n => printNumber(n, 2)).
```

- En el ejemplo anterior, se define un Action que toma como entrada 2 parámetros.
- Multiplica los parámetros y los muestra por pantalla.
- Al realizar esta acción sobre todos los elementos, nos muestra el resultado de la acción sobre cada miembro de la lista.

3. Delegados



Func

- El delegado Func representa un método que tiene un valor de retorno y puede aceptar entre 0 y 16 parámetros de entrada.
- El último tipo genérico especificado en Func representa el tipo de retorno.

3. Delegados

Func

- El delegado Func representa un método que tiene un valor de retorno y puede aceptar entre 0 y 16 parámetros de entrada.
- El último tipo genérico especificado en Func representa el tipo de retorno.

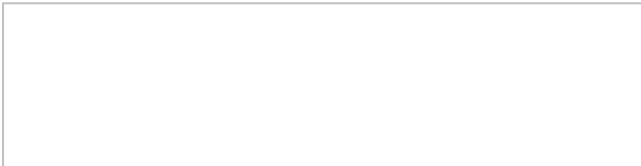
3. Delegados

Ejemplo

- En este ejemplo, nos declaramos una Func que acepta como entrada 2 int y devuelve un string en el formato especificado.
- Si ejecutamos el código, la salida que obtenemos es:

3. Delegados

```
1 Func<int, int, string> showMultiplication = (a, b) => strir  
2  
3 Console.WriteLine(showMultiplication(2, 4)).
```



- Func es común cuando se necesita realizar una operación y devolver un valor. También es frecuentemente utilizado en consultas LINQ.
- En este otro ejemplo únicamente toma como entrada un `int` y devuelve otro `int` en el que calculamos el cuadrado de la entrada:

3. Delegados

```
1 Func<int, int> square = x => x * x
2
3 Console.WriteLine(square(5)); // 25
```

Predicate

- Predicate es un tipo de delegado específico para métodos que devuelven un valor bool y aceptan un único parámetro de entrada.
- Se utiliza principalmente para verificaciones o filtros.

3. Delegados

```
1 Predicate<int> isEven = x => x % 2 == 0;
2
3 Console.WriteLine(isEven(4)); // Sal:
4 Console.WriteLine(isEven(5)); // Sal:
```

- Predicate es particularmente útil en métodos de colección como Find, FindAll, Exists, etc., donde se necesita una condición para evaluar elementos.
- En este ejemplo, pasamos el Predicate anterior como condición de la consulta FindAll. El resultado es una nueva lista con todos los objetos que cumplen el predicado.

3. Delegados

```
1 List<int> numbers = new List<int> {1, 2, 3, 4,  
2 List<int> evenNumbers = numbers.FindAll(isEven  
3  
4 evenNumbers.ForEach(Console.WriteLine); // Sal
```

Comparativa Func, Action, Predicate

3. Delegados

Delegado	Devuelve Valor	Parámetros	Uso Común
Action	No	0-16	Para realizar acciones sin retorno, como imprimir mensajes o actualizar valores
Func	Sí (cualquier tipo)	0-16	Para operaciones que necesitan devolver un resultado, como cálculos o transformaciones
Predica te	Sí (bool)	1	Para evaluaciones condicionales, como verificaciones o filtros

Ventajas de los Delegados

- **Flexibilidad:** Permiten implementar comportamientos personalizados y dinámicos, enviando métodos como argumentos a otros métodos.
- **Desacoplamiento:** Los delegados permiten ejecutar lógica sin conocer su implementación específica, lo que promueve el principio de "abstracción".

3. Delegados

Ventajas de los Delegados

- **Fundamento para Eventos:** Los eventos en C# están contruidos sobre delegados, lo que permite que los métodos suscritos a un evento se ejecuten cuando ocurre una acción específica.

En el siguiente punto vamos a ver los eventos y como estos utilizan los delegados.

3. Delegados

Vídeo práctico

3. Delegados

Vídeo práctico - código

3. Delegados

```
1  public delegate void ActionDelegate();
2  public delegate void ActionDelegateParams(int i, string
3
4  public delegate double SqrtFunc(int i);
5
6  public delegate bool Divisible3(int i);
7
8  public static void Main()
9  {
10     Action actionDelegate = () => Console.WriteLine("hol
11     Action<int, string, double> actionDelegateParams = (
12     Action<int> square = (i) => Console.WriteLine(i * i)
13
14     List<int> numbers = [ 1, 2, 3, 4, 5, 6 ];
15
16     numbers.ForEach(square);
17
18     square(7);
19     actionDelegate();
20     actionDelegateParams(3, "hola", 5.5);
21
```

- Los **eventos** son un mecanismo fundamental en C# que permite a los objetos comunicar cambios o notificaciones a otras partes del programa de manera flexible y eficiente.
- Estos son especialmente útiles en aplicaciones donde el flujo de la ejecución depende de acciones o respuestas del usuario, actualizaciones de datos, o cualquier otro cambio en el sistema.

4. Eventos

- En C#, un evento es esencialmente una **abstracción sobre un delegado**.
- Un evento:
 - Define un punto de notificación en una clase, llamado **publicador**.

4. Eventos

- Un evento:
- Permite que otras clases, llamadas **suscriptores**, se registren para recibir notificaciones cuando ocurre dicho evento.
- Se activa desde el publicador, llamando a los métodos registrados en los suscriptores cuando sucede la acción definida.

4. Eventos

Sintaxis de un evento

- La sintaxis para lanzar eventos se compone de 3 partes bien diferenciadas que se tienen que cumplimentar en la clase del publicador:
 - Definición del delegado
 - Declaración del evento
 - Invocación del evento

4. Eventos

- Para definir un evento, el primer paso es **definir el delegado**.
- Este actúa como una plantilla que especifica la firma de los métodos que manejarán el evento.
- Ejemplo:

4. Eventos

```
1 public delegate void ItemAddedHandler(string
```

- Una vez definido el delegado, debemos **declarar el evento** en sí.
- Este evento utiliza el tipo delegado que hemos definido precedido de la palabra event.
- Ejemplo:

4. Eventos

```
1 public event TtemAddedHandler On
```

- Una vez creado, podemos **invocar el evento** cuando queramos.
- Realizaremos esta invocación cuando se produzcan las **condiciones previstas** para nuestro evento (por ejemplo, en nuestro caso cuando se añade un nuevo item a una colección).
- Ejemplo:

4. Eventos

```
1 if(OnItemAdded != null)
2 {
3     OnItemAdded("Lavavajillas");
4 }
5
6 // Código equivalente al anterior,
7 // Usamos ? para ejecutar únicamente si
```

- El ejemplo completo del código podría estar implementado en una clase de gestión de almacén de la siguiente manera:

4. Eventos

```
1  public class Warehouse
2  {
3      public delegate void ItemAddedHandler(string item);
4      public event ItemAddedHandler OnItemAdded;
5
6      public List<string> Items { get; set; }
7
8      public void AddItem(string item)
9      {
10         Items.Add(item);
11
12         // Invocamos el evento cuando se añade
13         OnItemAdded?.Invoke(item);
14     }
```

Consumo de eventos

- Una vez que nuestro evento está definido, creado y es invocado por la clase publicadora, este puede ser consumido por las **clases suscriptoras**.
- Estas clases se suscriben a los eventos para ser informados de que ha ocurrido algo.

4. Eventos

- Para suscribirnos a un evento, utilizamos +=, seguido del método que va a manejar el evento.
- Para el ejemplo definido anteriormente, la suscripción y manejo sería:

4. Eventos

```
1 Warehouse warehouse = new Warehouse()  
2 Warehouse.OnItemAdded += Warehouse_It  
3  
4 ...  
5  
6 public void Warehouse_ItemAdded(strir  
7 {  
8     // Gestión del evento  
9 }
```

- Cada vez que la clase publicadora invoque el evento `OnItemAdded`, se ejecutará el método que lo maneja en las clases suscriptoras (`Warehouse_ItemAdded`)
- Las clases que se suscriben a eventos deben realizar la **desuscripción** de manera correcta para evitar problemas de rendimiento y fugas de memoria. Para ello utilizamos la expresión `-=` de manera similar que el `+=` en la suscripción.

4. Eventos

Vídeo práctico

4. Eventos

Vídeo práctico - código TemperatureSensor

4. Eventos

```
1  internal class TemperatureSensor
2  {
3      public delegate void TemperatureChanged(double temperature);
4
5      public event TemperatureChanged OnTemperatureChanged;
6
7      private Random _random = new Random(DateTime.Now.Millisecond);
8
9      public void StartReadings()
10     {
11         while (true)
12         {
13             var temp = 15 + _random.NextDouble() * 10;
14
15             OnTemperatureChanged?.Invoke(temp);
16
17             Thread.Sleep(1500);
18         }
19     }
20 }
```

Vídeo práctico - código Display

4. Eventos

```
1  internal class Display
2  {
3      private TemperatureSensor _sensor;
4
5      public Display()
6      {
7          _sensor = new TemperatureSensor();
8          _sensor.OnTemperatureChanged += _sensor_OnTemperat
9
10         _sensor.StartReadings();
11     }
12
13     private void _sensor_OnTemperatureChanged(double tempe
14     {
15         Console.WriteLine("Temperatura recibida: {0:F2}",
16     }
17 }
```

Vídeo práctico - código Program

4. Eventos

```
1 public static void Main()  
2 {  
3     Display display = new D
```

Eventos genéricos

- En C#, no siempre es necesario definir un delegado explícitamente cuando se usan eventos.
- **La mayoría de las veces** se puede utilizar el delegado genérico `EventHandler` o `EventHandler<TEventArgs>`, que es parte de la biblioteca estándar y cubre muchos casos de uso típicos de eventos.

4. Eventos

- EventHandler es un delegado genérico cuya firma es:
- Podemos declararnos nuestros eventos directamente como EventHandler, especificando opcionalmente el tipo de argumento.
- Se invocan pasando como parámetro el objeto que provoca su invocación (usualmente `this` o, en aplicaciones gráficas, un elemento de la interfaz) y un objeto del tipo especificado como argumento.

4. Eventos

```
public delegate void EventHandler(object? sender,
```

- Para el caso anterior, podríamos definir nuestro evento utilizando el delegado genérico `EventHandler` en vez de crear el delegado propio `ItemAddedHandler` de la siguiente manera:

4. Eventos

```
1  public class Warehouse
2  {
3      public event EventHandler<string> OnItemAdded
4
5      public List<string> Items { get; set; }
6
7      public void AddItem(string item)
8      {
9          Items.Add(item);
10
11         // Invocamos el evento cuando se a
12         OnItemAdded?.Invoke(this, item);
13     }
```

- Si no especificamos el tipo de EventHandler, se utiliza EventArgs en la invocación por defecto, que podremos extender para crear nuestros propios argumentos.

4. Eventos

```
1 public event EventHandler OnNewEvent;  
2  
3 ...  
4  
5 // Invocamos con un EventArgs vacío  
6 OnNewEvent?.Invoke(this, EventArgs.Empty);  
7  
8 // Invocamos con una nueva instancia de EventArgs  
9 OnNewEvent?.Invoke(this, new EventArgs());
```

- También podemos suscribirnos utilizando como método de manejo una **función lambda** directamente que cumpla con la firma requerida.

4. Eventos

```
1 public event ItemAddedHandler OnItemAdded;
2 public event EventHandler OnNewEvent;
3
4 ...
5
6 // Cuando llega el evento, ejecutamos la función anónima:
7 warehouse.OnItemAdded += (item) => Console.WriteLine("{0} añad
8
9 // Invocamos con una nueva instancia de EventArgs
10 publisher.OnNewEvent += (sender, args) => Console.WriteLine("E
```

Vídeo práctico

4. Eventos

Vídeo práctico - código TemperatureSensor

4. Eventos

```
1  internal class TemperatureSensor
2  {
3      public event EventHandler<double> OnTemperatureChang
4      public event EventHandler OnAlarmRaised;
5
6      private Random _random = new Random(DateTime.Now.Mil
7
8      public void StartReadings()
9      {
10         while (true)
11         {
12             var temp = 15 + _random.NextDouble() * 10;
13
14             OnTemperatureChanged?.Invoke(this, temp);
15
16             if(temp > 23)
17             {
18                 OnAlarmRaised?.Invoke(this, EventArgs.Em
19             }
```

Vídeo práctico - código Display

4. Eventos

```
1  internal class Display
2  {
3      private TemperatureSensor _sensor;
4
5      public Display()
6      {
7          _sensor = new TemperatureSensor();
8          _sensor.OnTemperatureChanged += _sensor_OnTemper
9          _sensor.OnAlarmRaised += _sensor_OnAlarmRaised;
10
11         _sensor.StartReadings();
12     }
13
14     private void _sensor_OnAlarmRaised(object? sender, E
15     {
16         Console.WriteLine("TEMPERATURA EXCEDIDA DETECTAD
17     }
18
19     private void _sensor_OnTemperatureChanged(object sen
20 }
```

- Al principio del curso vimos los conceptos de paradigmas de programación, entre ellos el de la **programación funcional**.
- Este es un enfoque en el que el código se centra en la creación y combinación de **funciones puras** para resolver problemas, en lugar de usar un estilo exclusivamente orientado a objetos o basado en el estado del programa.

5. Programación funcional en C#

- En C#, este paradigma se integra mediante expresiones lambda, delegados, LINQ, métodos de extensión y funciones de orden superior.
- Aunque C# es un lenguaje orientado a objetos, incluye muchas características funcionales que permiten adoptar prácticas de programación funcional, como la inmutabilidad y la composición de funciones.

5. Programación funcional en C#

Principios Clave de la Programación Funcional

- **Funciones Puras:** Una función pura es aquella cuyo resultado solo depende de sus parámetros de entrada y no tiene efectos secundarios (no modifica variables externas, no realiza operaciones de entrada/salida, etc.)

5. Programación funcional en C#

```
1 Func<int, int, int> add = (x, y) => x + y;  
2 int result = add(3, 5); // Siempre devolverá 8 si se le
```

- **Inmutabilidad:** En la programación funcional, se prioriza el uso de estructuras de datos inmutables, lo que significa que los valores y estados no se modifican una vez creados.
- En su lugar, se crean nuevos valores. Esto reduce errores y mejora la seguridad de los datos compartidos en aplicaciones concurrentes o multihilo.

5. Programación funcional en C#

- **Funciones de Orden Superior:** Son funciones que pueden recibir otras funciones como parámetros o devolverlas como resultados.
- Esto permite una mayor flexibilidad y modularidad, y se logra en C# usando delegados, expresiones lambda, y tipos Func y Action.

5. Programación funcional en C#

```
1 Func<int, int, int> multiply = (x, y  
2 int product = multiply(4, 5); // Dev
```

- **Expresiones Lambda:** Simplifican la escritura de funciones anónimas y se utilizan ampliamente en LINQ y otros contextos funcionales.
- Permiten definir rápidamente comportamientos y se usan con delegados o tipos `Func` y `Action`.

5. Programación funcional en C#

```
1 Func<int, bool> isEven = x => x % 2 == 0;
```

- **Composición de Funciones:** Permite combinar funciones pequeñas en otras más complejas.
- C# facilita la composición a través de métodos de extensión y encadenamiento de operaciones, como con LINQ.

5. Programación funcional en C#

```
1 var numbers = new List<int> { 1, 2, 3, 4,
2 var evenSquares = numbers.Where(x => x %
3                               .Select(x => x * x
4
5 // Filtra los números pares y los eleva a
```

Ventajas de la Programación Funcional

1. **Legibilidad y Mantenimiento:** La estructura de funciones concisas y específicas es fácil de leer y entender.
2. **Concurrente y Paralelizable:** La inmutabilidad y la ausencia de efectos secundarios facilitan la ejecución de código en paralelo.
3. **Pruebas y Depuración:** Las funciones puras son más fáciles de probar, ya que no dependen de estados externos.

5. Programación funcional en C#

- En este tema hemos visto las funciones lambda, los delegados y los eventos.
- Se trata de un tema con una complejidad elevada debido a que introducen un nuevo enfoque a la hora de abordar las tareas.
- Por ello, vamos a realizar un pequeño análisis sobre cada uno de los tipos, cuándo usarlo y qué ventajas e inconvenientes presenta.

6. Diferencias y ventajas de cada tipo

- Cada tipo de funcionalidad tiene un rol específico que aporta ventajas según el contexto:
- **Funciones lambda** para mejorar la concisión y la legibilidad.
- **Delegados** para la parametrización de métodos.
- **Eventos** para la comunicación entre objetos.

6. Diferencias y ventajas de cada tipo

Funciones Lambda

- Las **funciones Lambda** son expresiones concisas que simplifican la creación de funciones anónimas.
- **Ventajas:**
 - Facilitan la **legibilidad y simplicidad** de código, al evitar la necesidad de definir métodos largos.
 - Potencian la **concisión en LINQ** y consultas con colecciones, facilitando la programación funcional.

6. Diferencias y ventajas de cada tipo

Delegados

- Los **delegados** son tipos que definen una referencia a métodos que coinciden con una determinada firma.
- Su principal propósito es representar métodos como variables y parámetros, permitiendo pasar el comportamiento (método) como dato.
- Esto los hace ideales para la programación basada en *callbacks*, donde un método se pasa como argumento para ser ejecutado posteriormente.

6. Diferencias y ventajas de cada tipo

▸ **Ventajas:**

- Facilitan la **abstracción de comportamiento**, permitiendo que un método se parametrice en otros métodos.
- Son el núcleo de las **funciones lambda** y **eventos**, ya que se apoyan en delegados para establecer sus propias firmas.

6. Diferencias y ventajas de cada tipo

Eventos

- Los **eventos** son una forma de notificar a los suscriptores cuando ocurre una acción en un objeto.
- Se basan en delegados para definir qué tipo de métodos pueden suscribirse, pero también añaden una capa de control, limitando el acceso de manera que solo el propio objeto puede desencadenar el evento.

6. Diferencias y ventajas de cada tipo

▸ **Ventajas:**

- ▀ Permiten una comunicación **desacoplada** entre objetos.
- ▀ Controlan el acceso al delegado, de modo que otros objetos pueden suscribirse, pero no desencadenar el evento directamente.

6. Diferencias y ventajas de cada tipo

Comparativa y Uso Apropiado

6. Diferencias y ventajas de cada tipo

Tipo	Ideal para	Desventajas
Delegados	<i>Callbacks</i> , pasar métodos como datos	Puede volverse complejo en proyectos grandes
Eventos	Comunicación desacoplada	Solo se pueden activar en el publicador
Funciones Lambda	Concisión en consultas, LINQ	Menos adecuado para métodos complejos o reutilizables

- Aplicar buenas prácticas al trabajar con delegados, eventos y funciones lambda permite escribir código C# más seguro, modular y mantenible.
- Vamos a ver recomendaciones que no solo mejoran la eficiencia, sino que también evitan problemas de rendimiento, fugas de memoria y otros errores comunes.

7. Buenas prácticas para el uso de Lambdas, delegados y eventos

Buenas prácticas con Funciones Lambda

- **Usar lambdas para operaciones concisas:** Las expresiones lambda son ideales para realizar operaciones pequeñas y fácilmente comprensibles.
- Para operaciones más complejas, es mejor optar por métodos tradicionales para evitar el aumento de complejidad

7. Buenas prácticas para el uso de Lambdas, delegados y eventos

- **Evitar variables estáticas en expresiones lambda:** Las variables estáticas usadas en lambdas pueden generar comportamientos inesperados en entornos multi-hilo o asíncronos (los veremos en el próximo tema).
- **Especificar el tipo de datos cuando sea confuso:** Aunque las lambdas en C# infieren los tipos, a veces es conveniente especificarlos para mejorar la claridad:

7. Buenas prácticas para el uso de Lambdas, delegados y eventos

```
1 Func<int, int, int> multiply = (int x, int y,
```

Buenas prácticas con Delegados

- **Usar delegados genéricos cuando sea posible:** En lugar de definir delegados personalizados, utilizar `Action`, `Func` y `Predicate`, ya que simplifican el código y evitan la sobrecarga de declarar nuevos tipos.
- **Definir delegados solo si representan una intención clara:** Evitar el abuso de delegados personalizados si ya existen delegados predefinidos que cumplen con los requisitos.

7. Buenas prácticas para el uso de Lambdas, delegados y eventos

- **Utilizar expresiones lambda para simplificar asignaciones:** Las expresiones lambda mejoran la claridad del código, sobre todo en operaciones simples.
- **Documentar la intención del delegado:** Cuando se usan delegados personalizados, es útil documentar el propósito para facilitar la comprensión y mantenimiento del código.

7. Buenas prácticas para el uso de Lambdas, delegados y eventos

Buenas prácticas con Eventos

- **Comprobar la suscripción al evento antes de invocarlo:** Usar `?.Invoke()` es una forma segura de invocar eventos, ya que evita excepciones si no hay suscriptores.
- También podemos hacer la comprobación `if ... != null`. Si invocamos un evento que no tiene suscriptores, se producirá una excepción `NullReferenceException`.

7. Buenas prácticas para el uso de Lambdas, delegados y eventos

- **Desuscribirse de los eventos:** Especialmente en aplicaciones de larga duración o con múltiples instancias, es importante liberar la referencia a los eventos para evitar fugas de memoria.
- **Usar eventos solo en casos de necesidad de notificación:** Los eventos están diseñados para enviar notificaciones sobre el cambio de estado, por lo que es recomendable no utilizarlos en lugar de métodos o propiedades.

7. Buenas prácticas para el uso de Lambdas, delegados y eventos

Diapositiva final

En este tema, hemos visto el uso de funciones lambda, delegados y eventos, que permiten notificar sobre cambios en el programa y crear código que actúe ante estos cambios.

Speaker notes